IA for Cybersecurity — Lab 02

Password Analysis

Detecting a shared default password in a corporate password audit

3rd Year — Second Cycle — Option CyS

Student: Manel Mostefaoui

Academic year 2026–2027 · October 2026

Deliverables: Lab02_Password_Analysis.ipynb + this report

Contents

1. Introduction and objective
2. Use case and dataset
3. Methodology
4. Exploratory analysis
5. Visualizing the anomaly
6. Detection rule
7. Evaluation
8. Main findings
9. Limitations and recommendations
10. Conclusion

Appendix — Running the notebook

1. Introduction and objective

Password policies usually focus on complexity: a minimum length and a mix of character classes. In practice, attackers rarely brute-force passwords at random. They try what people actually choose, and they exploit passwords that are shared between accounts. A single weak password reused on many accounts turns one successful guess into dozens of compromised users.

The objective of this lab is to apply data visualization and analysis techniques to the **password analysis** use case: explore a password-audit dataset, inject one realistic anomaly, make it visible with charts, turn the visual insight into a simple automatic detection rule, and evaluate that rule on both anomalous and normal data.

2. Use case and dataset

2.1 Scenario

A mid-sized company runs an **authorized quarterly password audit**. The password hashes of all Active Directory accounts are exported and processed with a cracking tool (for example hashcat) in order to measure the real strength of employee passwords. The audit output is one row per account. Since no real credentials may be used, the dataset is **simulated** in Python with fixed random seeds, so every result in this report is reproducible.

2.2 Fields

Column	Description
`username`	Account name in firstname.lastname format
`department`	IT, HR, Finance, Sales, Operations, Legal, Marketing, R&D
`account_created`	Account creation date
`pwd_last_set`	Last password change (Active Directory attribute pwdLastSet)
`password`	Plaintext recovered by the cracking tool (exists only in an authorized audit)
`pwd_hash`	Unsalted SHA-256 hash, standing in for the Windows NT hash
`is_injected`	Ground-truth label, hidden from the detection rule and used only for evaluation

2.3 How normal passwords are generated

Normal users choose passwords following habits that are well documented in breach studies. All generated passwords satisfy the company policy: **at least 8 characters and at least 3 of the 4 character classes** (upper case, lower case, digit, symbol).

Habit	Example	Share
Word + year (+ symbol)	`Mouloudia2019!`	38 %
Own first name + date	`Karim0312!`	17 %

Habit	Example	Share
Leet-speak word + digits	`Dr@g0n47`	14 %
Passphrase	`Tiger-Coffee-Blue7`	13 %
Random (password manager)	`q7#Lm…`	12 %
Well-known compliant password	`P@ssw0rd123`	6 %

2.4 Injected anomaly

On **1 September 2026**, a cohort of **40 new employees** is onboarded. The helpdesk creates all accounts within one hour with the same default password, `Welcome@2026,` and forgets to tick "User must change password at next logon". Six weeks later, at the audit date (15 October 2026), none of these passwords has been changed. This is a common real-world finding in internal penetration tests.

Note that `Welcome@2026` is fully compliant with the complexity policy (12 characters, 4 classes). A policy check alone would never catch it.

2.5 Three audit exports

Dataset	Accounts	Purpose	Anomaly
baseline (April 2026)	1,150	Learn what normal password sharing looks like; calibrate the rule threshold	No
current (15 Oct 2026)	1,240	Audit under investigation	Yes — 40 accounts
clean (independent)	1,200	Measure false positives on purely normal data	No

Calibrating the threshold on a separate dataset avoids tuning the rule on the very data it is evaluated on.

3. Methodology

The analysis is done in **Python** (pandas, NumPy, Matplotlib, Seaborn) in a Jupyter notebook, plus **zxcvbn**, the open-source password-strength estimator published by Dropbox. Python was preferred over Kibana because the data is a single tabular export and the strength features require custom code. The pipeline follows six steps:

1. **Generate** the three audit exports (section 2).
2. **Extract features** for each account: length, character classes, policy compliance, zxcvbn score (0 = very weak … 4 = very strong), estimated offline crack time at 10^{10} guesses/s (a realistic GPU speed for NT hashes), pattern flags (contains a year, contains the user's own first name), the age of the password, and `share_count`, the number of accounts having the same hash.
3. **Explore** the normal population: distributions of length and strength, and strength per department.
4. **Visualize the anomaly** from two independent angles: password sharing and password-set timing.

5. **Define a detection rule** that uses only the hash column and calibrate its threshold on the baseline audit.
6. **Evaluate** the rule at account level against the ground truth on the current audit and on the clean audit, and test its sensitivity to the threshold.

4. Exploratory analysis

Indicator (current audit, 1,240 accounts)	Value
Policy-compliant passwords	100 %
Weak passwords (zxcvbn score ≤ 2)	60.6 %
Crackable offline in under 1 second	73.0 %
Contain a year (19xx / 20xx)	43.5 %
Contain the user's own first name	17.4 %
Median length	11 characters
Normal accounts sharing their password with at least one other account	113

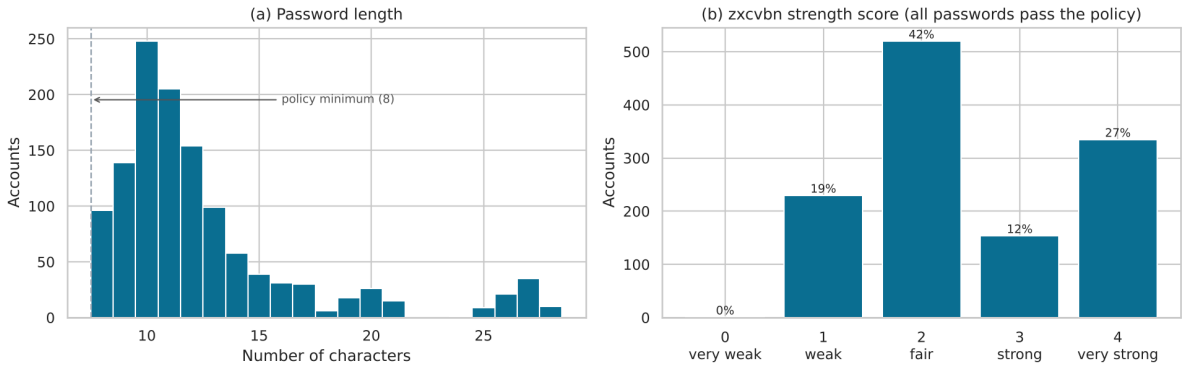


Figure 1 — *Password length (a) and zxcvbn strength score (b) for all accounts of the current audit*

Observation. Every password passes the policy, and most are 9 to 13 characters long. Yet 61 % are rated weak or fair by zxcvbn and 73 % would fall in under a second against an offline attack, because they follow predictable structures (word + year, first name + date). The second peak at 25–28 characters corresponds to passphrases, the only habit that gives both length and strength.

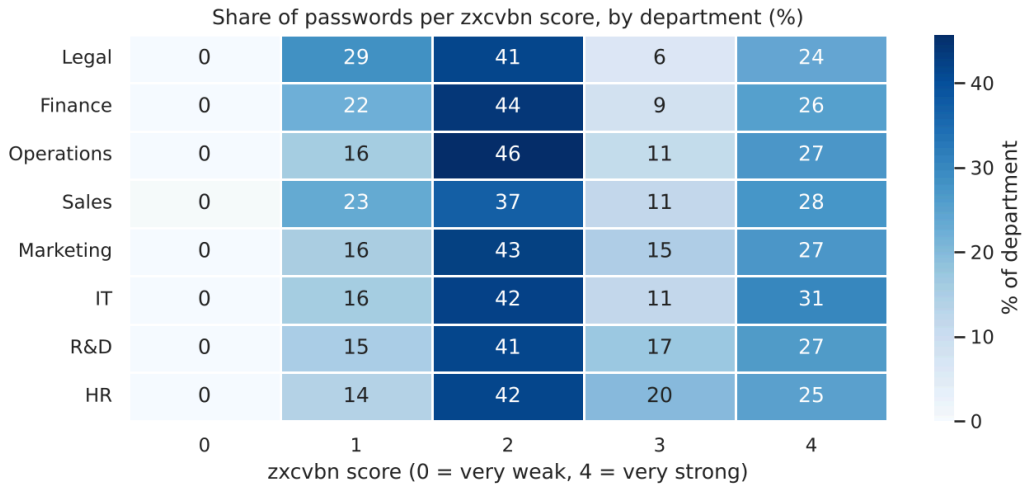


Figure 2 — *Distribution of zxcvbn scores per department (each row sums to 100 %)*

Observation. The pattern is similar in every department: score 2 dominates everywhere. Legal and Finance have the highest share of weak passwords (about 70 % and 66 % at score ≤ 2), HR and R&D the lowest. Legal is a small department, so its figure is less stable. These differences can guide where awareness training is most useful, but no department stands out as anomalous on strength alone.

5. Visualizing the anomaly

5.1 Password sharing

In a normal population, a few users independently pick the same famous password, so small clusters of 2 to 9 accounts sharing a hash are expected. Figure 3 ranks the 15 most shared hashes. Only hash prefixes are displayed: the analyst does not need the plaintext to see reuse.

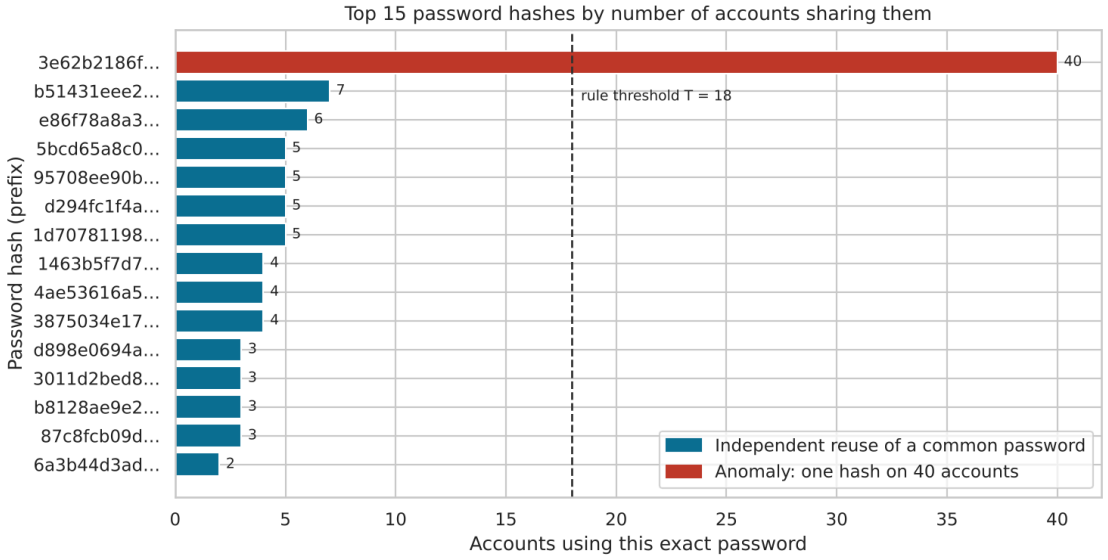


Figure 3 — *Top 15 password hashes by number of accounts sharing them; the dashed line is the rule threshold*

Where the anomaly is: one hash, `3e62b2186f`…, is used by **40 accounts**, more than five times the largest normal cluster (7 accounts). Once cracked, it reveals `Welcome@2026`.

5.2 Password-set timing

With a 90-day rotation policy, password changes are spread evenly over time (about 9 per day). Figure 4 counts password-set events per day over the 120 days preceding the audit.

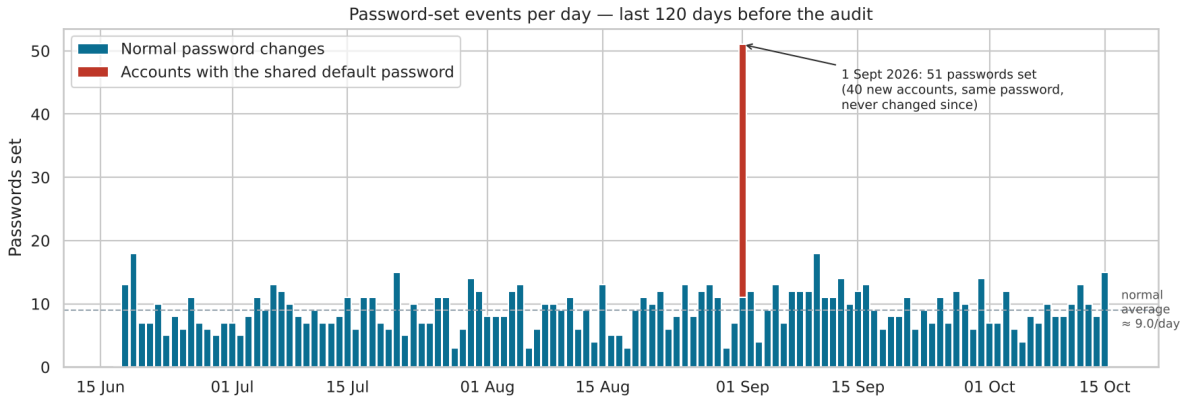


Figure 4 — *Password-set events per day; the red part is the 40 accounts carrying the shared default password*

Where the anomaly is: on **1 September 2026**, 51 passwords were set instead of the usual ~9. The extra 40 events were all created between 09:00 and 09:58, all carry the same hash, and none was followed by a password change. The two views are independent: one comes from the hash column, the other from the timestamps. Together they point to a bulk onboarding where the first-logon password change was not enforced. The affected accounts are spread over all eight departments (Sales 9, Finance 7, R&D 7, Operations 7, Marketing 5, HR 3, IT 1, Legal 1), which is consistent with a company-wide hiring wave rather than a single team.

6. Detection rule

> **Rule — "mass password reuse"**
> Flag every account whose password hash is shared by **T or more** accounts in the same audit.
> Chosen threshold: **T = 18**.

Equivalent SQL, which can be scheduled in a SIEM or run on the hash export:

```
SELECT username FROM audit
WHERE pwd_hash IN (SELECT pwd_hash FROM audit
                   GROUP BY pwd_hash HAVING COUNT(*) >= 18);
```

Why it works without cracking. Windows NT hashes are not salted, so two identical passwords always produce identical hashes. The rule only counts hashes and can run on the raw export, before any plaintext is recovered.

Choosing T. Some sharing is normal. On the baseline audit, the five largest normal clusters contain 9, 8, 7, 6 and 6 accounts. The threshold is set to **T = max(10, 2 × largest baseline cluster) = 18**. This leaves a safety margin above ordinary coincidences while still firing on any administrative or deliberate reuse.

7. Evaluation

The rule output is compared with the ground-truth label at account level. A true positive (TP) is an injected account that is flagged; a false positive (FP) is a normal account that is flagged.

Dataset	Total	Anom.	TP	FP	FN	TN	Prec.	Recall	FPR
current (with anomaly)	1,240	40	40	0	0	1,200	1.00	1.00	0.0 %
clean (normal only)	1,200	0	0	0	0	1,200	—	—	0.0 %

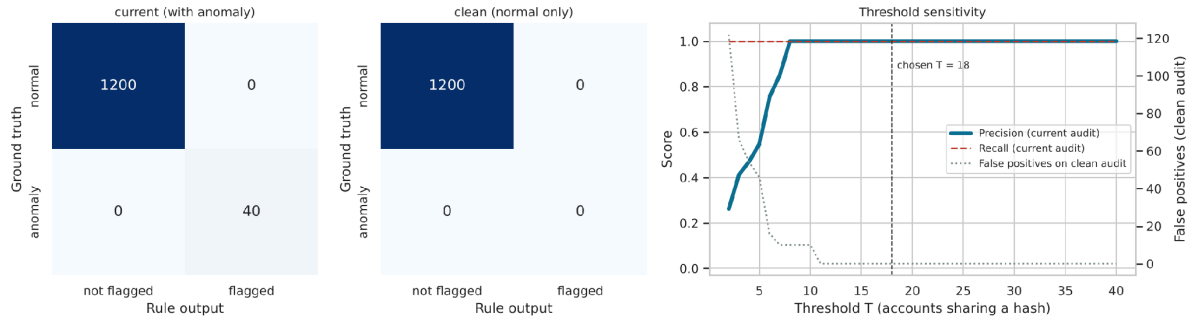


Figure 5 — *Confusion matrices on the current and clean audits (left) and threshold sensitivity (right)*

Results. On the anomalous audit, the rule flags all 40 injected accounts and no normal account (precision = recall = 1.00). On the clean audit, it raises no alert at all (false-positive rate 0 %).

Robustness. The sweep on the right of Figure 5 shows that recall stays at 1.00 for every threshold up to 40, while precision only reaches 1.00 from T = 8, and false positives on the clean audit disappear from T = 11. Every value of T between 11 and 40 gives a perfect result, so the chosen value of 18 sits well inside a wide safe range and does not depend on a lucky choice. Thresholds below 8 would flag many normal users who independently picked famous passwords (about 120 false positives at T = 2).

8. Main findings

1. **Complexity rules give a false sense of security.** 100 % of passwords comply with the policy, yet 61 % are weak and 73 % can be cracked offline in under a second. The anomalous password `Welcome@2026` is itself compliant.
2. **Predictable patterns dominate.** 43.5 % of passwords contain a year and 17.4 % contain the user's own first name: exactly the patterns that cracking rule-sets try first.
3. **The anomaly is visible from two independent angles:** one hash on 40 accounts (Figure 3) and a burst of 40 password-set events on 1 September 2026 never followed by a change (Figure 4).
4. **A one-line rule detects it reliably.** With T = 18, the rule finds all 40 accounts with zero false positives on both the anomalous and the clean audit, and the result holds for any threshold from 11 to 40.
5. **Ordinary reuse is still a risk.** 113 normal accounts share their password with at least one other account (clusters of 2 to 7). They are below the threshold and are not the injected anomaly, but a single password-spraying attempt with a common password would compromise each cluster at once.

9. Limitations and recommendations

9.1 Limitations

- The data is simulated. Real password habits depend on language, country and company culture, so the exact percentages would differ.
- The rule relies on **unsalted** hashes, which is true for Windows NT hashes. With salted algorithms (bcrypt, Argon2) identical passwords give different hashes and reuse can only be found after cracking.
- A cohort smaller than T (for example 5 accounts) would be missed. A second condition, “at least 3 accounts sharing a hash **and** created within the same hour”, would catch small cohorts while keeping false positives low.
- The evaluation uses one injected scenario. A broader test would vary the cohort size and mix several anomaly types.

9.2 Recommendations

- Enforce “change password at next logon” and generate a unique random initial password for every new account.
- Deploy a banned-password list (for example Microsoft Entra Password Protection) including company-specific words and the current year.
- Run the reuse rule automatically after every account-creation batch and in each quarterly audit.
- Encourage passphrases and password managers, which are the only habits that produced strong passwords in this analysis.

10. Conclusion

This lab showed how visualization turns a flat password-audit export into actionable insight. Two simple charts made a shared default password on 40 new accounts obvious, and that visual insight translated directly into a one-line detection rule working on hashes alone. Evaluated on anomalous and normal data, the rule detected every affected account without a single false positive, across a wide range of thresholds. The analysis also exposed a broader weakness: a policy based only on complexity lets predictable and shared passwords through.

Appendix — Running the notebook

```
pip install pandas numpy matplotlib seaborn zxcvbn jupyter
jupyter notebook Lab02_Password_Analysis.ipynb
```

Run all cells from top to bottom. The notebook generates the data, saves the five figures to a `figures/` folder and prints every number quoted in this report.